

后端服务部署文档

后端服务部署文档

1. 文档概述
2. 后端部署总体架构
3. 部署环境要求
 - 3.1 服务器环境
 - 3.2 基础软件
 - 3.3 目录结构
4. 代理软件部署
 - 4.1 代理服务位置
 - 4.2 下载项目
 - 4.3 启动程序
 - 4.4 重启程序
 - 4.5 切换节点
 - 4.5.1 创建start-and-switch.sh脚本
 - 4.5.2 运行脚本
 - 4.5.3 测试
 - 4.6 提示
 - 4.6.1 谨慎选择其他代理软件
 - 4.6.2 选择合适的机场
5. 后端服务与端口对应关系
6. Docker Compose 自动化部署
 - 6.1 5.1 docker-compose.yml 示例
 - 6.2 常用部署命令
7. CI/CD 工作流部署
 - 7.1 CI/CD 流程说明
 - 7.2 GitHub Actions 示例
 - 7.3 Secret 配置说明
8. 自动化部署脚本
 - 8.1 `deployment.sh` 示例
9. Nginx / API Gateway 转发说明
10. Docker 调试与故障排查
 - 10.1 查看容器状态
 - 10.2 查看容器日志
 - 10.3 进入容器内部
 - 10.4 测试宿主机端口
 - 10.5 测试容器内部服务
 - 10.6 查看端口占用
 - 10.7 查看容器详细信息
 - 10.8 常见问题
11. 回滚方案
 - 11.1 使用旧版本镜像回滚
 - 11.2 使用 Git 回滚配置文件
 - 11.3 回滚注意事项
12. 安全与维护建议

13. 附录：常用命令

13.1 Docker Compose 常用命令

13.2 Docker 容器常用命令

13.3 Docker 镜像常用命令

13.4 端口与网络排查命令

13.5 Nginx 常用命令

13.6 系统服务与防火墙命令

13.7 常用健康检查命令

13.8 部署脚本执行命令

13.9 常见故障快速定位命令

1. 文档概述

本文档用于说明智慧幕墙系统后端服务的部署流程、CI/CD 自动化 workflow、Docker Compose 编排方式、服务端口映射关系以及常见 Docker 调试方法。

本文档适用于以下人员：

- 后端开发人员
- 运维部署人员
- 测试人员
- 项目维护人员

本文档覆盖内容包括：

- 后端服务部署环境准备
- Docker Compose 自动化部署
- GitHub Actions CI/CD 工作流
- 后端服务端口对应关系
- Nginx / API Gateway 转发说明
- Docker 容器调试与故障排查

2. 后端部署总体架构

系统后端采用容器化部署方式，各后端服务通过 Docker 镜像进行打包，并使用 Docker Compose 统一编排、启动和管理。

整体部署结构如下：

用户请求首先进入 Nginx/API Gateway，由网关根据请求路径将流量转发到不同的后端服务。各后端服务运行在独立容器中，通过固定端口暴露服务能力。业务服务、算法服务、评估服务等模块相互独立，便于后续扩展、升级和故障隔离。

部署流程主要包括：

1. 开发人员提交代码到 GitHub 仓库；

- 2. GitHub Actions 触发 CI/CD 工作流;
- 3. 工作流完成代码拉取、依赖安装、镜像构建和镜像推送;
- 4. 服务器拉取最新镜像;
- 5. Docker Compose 根据配置文件重新创建或更新容器;
- 6. Nginx/API Gateway 将外部请求转发到对应后端服务。

3. 部署环境要求

3.1 服务器环境

项目	要求
操作系统	Ubuntu / Debian / CentOS 等 Linux 系统
CPU	2 核及以上
内存	2GB 及以上
Docker	建议使用 Docker Engine 最新稳定版
Docker Compose	建议使用 Docker Compose v2
网络	服务器需要能够访问 Docker Hub 或私有镜像仓库

Table 1

3.2 基础软件

服务器需要提前安装以下组件:

- Docker
- Docker Compose
- Git
- Nginx
- curl
- vim / nano
- systemctl 相关服务管理工具

3.3 目录结构

建议在服务器中统一使用如下目录结构:

```
1 | /home/ecs-user/Intelligent_Curtain_wall/
2 |   ├── docker-compose.yml
3 |   ├── .env
4 |   ├── nginx/
5 |   |   └── default.conf
6 |   ├── logs/
7 |   ├── scripts/
8 |   |   ├── deploy.sh
9 |   |   └── rollback.sh
10 |   └── services/
```

Fence 1

4. 代理软件部署

4.1 代理服务位置

位于服务器8.159.143.133和8.153.161.229的ecs-user/clash-for-linux文件夹中

4.2 下载项目

下载项目

```
1 | git clone https://github.com/wnlen/clash-for-linux.git
```

Fence 2

进入到项目目录，编辑 `.env` 文件，修改变量 `CLASH_URL` 的值。

```
1 | cd clash-for-linux
2 | vim .env
```

Fence 3

注意： `.env` 文件中的变量 `CLASH_SECRET` 为自定义 Clash Secret，值为空时，脚本将自动生成随机字符串。

需要自己添加url订阅链接

```
1 | # Clash 订阅地址
2 | export CLASH_URL='更改为你的clash订阅地址'
3 | export CLASH_SECRET=''
4 | export CLASH_HEADERS='User-Agent: Clashforwindows/0.20.39'
5 |
6 | # Clash 监听配置
7 | export CLASH_HTTP_PORT=7890
8 | export CLASH_SOCKS_PORT=7891
9 | export CLASH_REDIR_PORT=7892
10 | export CLASH_LISTEN_IP=0.0.0.0
11 | export CLASH_ALLOW_LAN=true
12 |
13 | # External Controller (RESTful API) 配置
```

```
14 export EXTERNAL_CONTROLLER_ENABLED=true
15 export EXTERNAL_CONTROLLER=0.0.0.0:9090
```

Fence 4

4.3 启动程序

直接运行脚本文件 `start.sh`

- 进入项目目录

```
1 $ cd clash-for-linux
```

Fence 5

- 运行启动脚本

```
1 $ sudo bash start.sh
2
3 正在检测订阅地址...
4 Clash订阅地址可访问! [ OK ]
5
6 正在下载Clash配置文件...
7 配置文件config.yaml下载成功! [ OK ]
8
9 正在启动Clash服务...
10 服务启动成功! [ OK ]
11
12 Clash Dashboard 访问地址: http://<ip>:9090/ui
13 Secret: xxxxxxxxxxxxxx
14
15 请执行以下命令加载环境变量: source /etc/profile.d/clash.sh
16
17 请执行以下命令开启系统代理: proxy_on
18
19 若要临时关闭系统代理, 请执行: proxy_off
20
```

Fence 6

```
1 $ source /etc/profile.d/clash.sh
2 $ proxy_on
```

Fence 7

- 检查服务端口

```

1  $ netstat -tln | grep -E '9090|789.'
2  tcp        0      0 127.0.0.1:9090      0.0.0.0:*           LISTEN
3  tcp6       0      0 :::7890             :::*                LISTEN
4  tcp6       0      0 :::7891             :::*                LISTEN
5  tcp6       0      0 :::7892             :::*                LISTEN

```

Fence 8

- 检查环境变量

```

1  $ env | grep -E 'http_proxy|https_proxy'
2  http_proxy=http://127.0.0.1:7890
3  https_proxy=http://127.0.0.1:7890

```

Fence 9

以上步骤如果正常，说明服务clash程序启动成功，现在就可以体验高速下载github资源了。

4.4 重启程序

重新进行以上三步即可

```

1  sudo bash start.sh
2  source /etc/profile.d/clash.sh
3  proxy_on

```

Fence 10

4.5 切换节点

由于有些节点可能不可用，因此开发者新增了一个切换节点的功能

4.5.1 创建start-and-switch.sh脚本

将以下代码复制进脚本中

```

1  #!/usr/bin/env bash
2  set -euo pipefail
3
4  CLASH_DIR="/home/ecs-user/clash-for-linux"
5  GROUP_NAME="🌍 国外流量"
6  NODE_NAME="美国G | 0.1x-下载专用"
7
8  cd "$CLASH_DIR"
9
10 echo "[1/6] Restart Clash"
11 sudo bash shutdown.sh || true
12 sleep 2
13

```

```
14 START_LOG="$(mktemp)"
15 sudo bash start.sh | tee "$START_LOG"
16
17 echo "[2/6] wait for Clash API"
18 for i in $(seq 1 30); do
19     if ss -lntp | grep -q ':9090'; then
20         echo "Clash API is ready"
21         break
22     fi
23
24     if [ "$i" -eq 30 ]; then
25         echo "Clash API did not become ready"
26         exit 1
27     fi
28
29     sleep 1
30 done
31
32 echo "[3/6] Read Clash API secret"
33 SECRET="$(awk '/Secret:/ {print $2}' "$START_LOG" | tail -n1)"
34
35 if [ -z "$SECRET" ]; then
36     echo "Failed to read Clash Secret from start.sh output"
37     exit 1
38 fi
39
40 echo "[4/6] Switch node: $GROUP_NAME -> $NODE_NAME"
41
42 GROUP_NAME="$GROUP_NAME" NODE_NAME="$NODE_NAME" CLASH_SECRET="$SECRET" python3 -
43 <<'PY'
44 import json
45 import os
46 import urllib.parse
47 import urllib.request
48
49 group = os.environ["GROUP_NAME"]
50 node = os.environ["NODE_NAME"]
51 secret = os.environ["CLASH_SECRET"]
52
53 url = "http://127.0.0.1:9090/proxies/" + urllib.parse.quote(group, safe="")
54 data = json.dumps({"name": node}).encode()
55
56 headers = {
57     "Content-Type": "application/json",
58     "Authorization": "Bearer " + secret,
59 }
```

```

60 req = urllib.request.Request(url, data=data, method="PUT", headers=headers)
61
62 # 访问 Clash 控制 API 时不要走系统代理, 否则可能导致 401 或异常转发
63 opener = urllib.request.build_opener(urllib.request.ProxyHandler({}))
64
65 with opener.open(req, timeout=15) as resp:
66     print("OK", resp.status)
67 PY
68
69 echo "[5/6] Enable shell proxy"
70 source /etc/profile.d/clash.sh
71 proxy_on
72
73 echo "[6/6] Test proxy"
74 curl -I --connect-timeout 15 -x http://127.0.0.1:7890 https://www.google.com
75
76 echo "[OK] Clash started, node switched, proxy enabled."
77

```

Fence 11

4.5.2 运行脚本

```

1 | sudo bash start-and-switch.sh

```

Fence 12

4.5.3 测试

```

1 | curl -I www.google.com

```

Fence 13

若出现“200 OK”标志即为成功

4.6 提示

4.6.1 谨慎选择其他代理软件

由于不同代理软件所涉及的代理协议不同, 有的代理协议会被阿里云服务器识别而禁止用户翻墙, 所以选择正确的代理软件和代理协议是成功的第一步。

4.6.2 选择合适的机场

由于云服务器IP极易被识别的特性和机场要防止被云服务器“二次中转”所造成的商业滥用, 换句话说, 怕用户拿服务器去给别人当中转机场, 所以比较正规的梯子往往都会识别大型的服务器平台, 如阿里云。因此使用一个限制宽松, 不会特意识别服务器的机场是十分有必要的, 建议多换几个同学的机场试试。除此之外, 由于docker镜像往往比较大(可能达几G), 有些机场还会限制单次下载的流量, 所以在阿里云服务器上配置合适的机场并非易事。

5. 后端服务与端口对应关系

系统中的后端服务通过 Docker 容器运行，每个服务在宿主机上暴露一个固定端口，Nginx/API Gateway 根据请求路径将流量转发到对应服务。

服务名称	容器名称	镜像名称	宿主机端口	容器端口	功能说明
用户认证服务	user-authentication	clearpool/intelligent-curtain-wall:user-authentication-new	8008	8000	用户登录、注册、权限认证
裂缝检测服务	crack-detection	clearpool/intelligent-curtain-wall:crack-detection	18001	8080	石材裂缝识别与检测
幕墙评估服务	resilience-assessment	clearpool/intelligent-curtain-wall:resilience-assessment	18005	8080	幕墙韧性与安全评估
污渍检测服务	stain-detection	clearpool/intelligent-curtain-wall:stain-detection	18007	8080	石材污渍识别与分析
振动检测服务	vibration-detection	clearpool/intelligent-curtain-wall:vibration-detection	18009	8080	幕墙振动数据分析

Table 2

端口访问示例：

```
1  curl http://localhost:8008/api/account/login
2  curl http://localhost:18001/
3  curl http://localhost:18005/
4  curl http://localhost:18007/
5  curl http://localhost:18009/
```

Fence 14

说明：

- 宿主机端口用于服务器外部或 Nginx 访问；
- 容器端口为服务在容器内部监听的端口；
- 不建议直接暴露数据库端口到公网；
- 对外访问应优先通过 Nginx/API Gateway 统一入口。

6. Docker Compose 自动化部署

系统使用 Docker Compose 对多个后端服务进行统一编排。通过一个 `docker-compose.yml` 文件，可以同时完成镜像拉取、容器启动、端口映射、环境变量注入和服务重启策略配置。

6.1 5.1 docker-compose.yml 示例

```
1  services:
2      user-authentication:
3          image: clearpool/intelligent-curtain-wall:user-authentication-new
4          container_name: user-authentication
5          restart: always
6          ports:
7              - "8008:8000"
8          env_file:
9              - .env
10         networks:
11             - curtainwall-net
12
13     crack-detection:
14         image: clearpool/intelligent-curtain-wall:crack-detection
15         container_name: crack-detection
16         restart: always
17         ports:
18             - "18001:8080"
19         volumes:
20             - /home/ecs-user/Intelligent_Curtain_Wall/crack-
21               detection:/segformerProject/model
22         networks:
23             - curtainwall-net
24
25     stain-detection:
26         image: clearpool/intelligent-curtain-wall:stain-detection
27         container_name: stain-detection
28         restart: always
29         ports:
30             - "18007:8080"
31         networks:
32             - curtainwall-net
33
34     resilience-assessment:
35         image: clearpool/intelligent-curtain-wall:resilience-assessment
36         container_name: resilience-assessment
37         restart: always
38         ports:
39             - "18005:8080"
40         networks:
```

```
40     - curtainwall-net
41
42     vibration-detection:
43         image: clearpool/intelligent-curtain-wall:vibration-detection
44         container_name: vibration-detection
45         restart: always
46         ports:
47             - "18009:8080"
48         networks:
49             - curtainwall-net
50
51     networks:
52         curtainwall-net:
53             driver: bridge
```

Fence 15

6.2 常用部署命令

启动全部服务：

```
1 | docker compose up -d
```

Fence 16

停止全部服务：

```
1 | docker compose down
```

Fence 17

拉取最新镜像：

```
1 | docker compose pull
```

Fence 18

重新部署：

```
1 | docker compose pull
2 | docker compose up -d
```

Fence 19

查看运行状态：

```
1 | docker compose ps
```

Fence 20

7. CI/CD 工作流部署

系统使用 GitHub Actions 实现 CI/CD 自动化部署。开发人员将代码推送到指定分支后，GitHub Actions 自动执行构建、镜像推送和远程服务器部署流程。

7.1 CI/CD 流程说明

CI/CD 工作流主要包括以下步骤：

1. 监听指定分支代码提交；
2. 拉取项目源代码；
3. 配置 Docker Build 环境；
4. 登录 Docker Hub 或私有镜像仓库；
5. 构建后端服务 Docker 镜像；
6. 推送镜像到镜像仓库；
7. 通过 SSH 连接远程服务器；
8. 在服务器上执行部署脚本；
9. 拉取最新镜像并重启 Docker Compose 服务。

7.2 GitHub Actions 示例

```
1  name: CI/CD Pipeline
2
3  on:
4    workflow_call:
5      inputs:
6        image-tag:
7          description: 'Docker image tag'
8          required: true
9          type: string
10     secrets:
11       WORKFLOW_PRIVATE_KEY:
12         required: true
13
14   jobs:
15     ci-cd-pipeline:
16       runs-on: ubuntu-latest
17
18       steps:
19         - name: Checkout the code
20           uses: actions/checkout@v3
21           with:
22             fetch-depth: 0
23             ref: ${github.ref }
```

```
24
25     - name: Set up SSH
26       run: |
27         mkdir -p ~/.ssh
28         echo "${{ secrets.WORKFLOW_PRIVATE_KEY }}" > ~/.ssh/id_rsa
29         chmod 600 ~/.ssh/id_rsa
30         ssh-keyscan github.com >> ~/.ssh/known_hosts
31
32     - name: Clone secrets repository
33       run: |
34         git clone git@github.com:Intelligent-Curtain-wall/.secrets.git
35         if [ ! -d ".secrets" ]; then
36           echo "Error: .secrets directory not found!"
37           exit 1
38         fi
39
40     - name: Log in to Docker Hub
41       run: |
42         echo "$(cat .secrets/DOCKER_PASSWORD)" | docker login --username "$(cat
43 .secrets/DOCKER_USERNAME)" --password-stdin
44
45     - name: Build and push Docker image
46       uses: docker/build-push-action@v4
47       with:
48         context: ./
49         file: ./Dockerfile
50         push: true
51         tags: docker.io/clearpool/intelligent-curtain-wall:${{ inputs.image-tag }}
52
53
54
55     - name: Connect to remote server and execute deployment script
56       run: |
57         mv .secrets/MATPOOL_SSH_PRIVATE_KEY ~/.ssh/matpool_id_rsa
58         chmod 600 ~/.ssh/matpool_id_rsa
59
60         SSH_PORT="$(cat .secrets/MATPOOL_SSH_PORT)"
61         SSH_USER="$(cat .secrets/MATPOOL_SSH_USERNAME)"
62         SSH_HOST="$(cat .secrets/MATPOOL_SSH_HOST)"
63
64         ssh -i ~/.ssh/matpool_id_rsa \
65           -p "$SSH_PORT" \
66           -o StrictHostKeyChecking=accept-new \
67           -o ConnectTimeout=20 \
68           -o ServerAliveInterval=20 \
69           -o ServerAliveCountMax=3 \
```

```

70     "$SSH_USER@$SSH_HOST" 'bash -s' <<'REMOTE_SCRIPT'
71 set -euo pipefail
72
73 CLASH_DIR="/home/ecs-user/clash-for-linux"
74 PROJECT_DIR="/home/ecs-user/Intelligent_Curtain_wall"
75
76 CLASH_GROUP="🌐 国外流量"
77 CLASH_NODE="美国G | 0.1x-下载专用"
78
79 echo "[1/7] Restart Clash proxy"
80 cd "$CLASH_DIR"
81 sudo bash shutdown.sh || true
82 sleep 2
83
84 sudo bash start.sh | tee /tmp/clash-start.log
85
86 echo "[2/7] wait for Clash ports"
87 for i in $(seq 1 30); do
88     if ss -lntp | grep -q ':7890' && ss -lntp | grep -q ':9090'; then
89         echo "Clash proxy and API are ready"
90         break
91     fi
92
93     if [ "$i" -eq 30 ]; then
94         echo "Clash did not become ready in time"
95         tail -n 100 /tmp/clash-start.log || true
96         exit 1
97     fi
98
99     sleep 1
100 done
101
102 echo "[3/7] Read Clash API secret"
103 CLASH_SECRET="$(awk '/Secret:/ {print $2}' /tmp/clash-start.log | tail -
n1)"
104
105 if [ -z "$CLASH_SECRET" ]; then
106     echo "Failed to read Clash Secret from startup log"
107     exit 1
108 fi
109
110 echo "[4/7] Switch Clash to Global mode and select node"
111 CLASH_GROUP="$CLASH_GROUP" CLASH_NODE="$CLASH_NODE"
CLASH_SECRET="$CLASH_SECRET" python3 - <<'PY'
112     import json
113     import os
114     import urllib.parse

```

```

115     import urllib.request
116
117     group = os.environ["CLASH_GROUP"]
118     node = os.environ["CLASH_NODE"]
119     secret = os.environ["CLASH_SECRET"]
120
121     headers = {
122         "Content-Type": "application/json",
123         "Authorization": "Bearer " + secret,
124     }
125
126     opener = urllib.request.build_opener(urllib.request.ProxyHandler({}))
127
128     def request(method, path, body=None):
129         url = "http://127.0.0.1:9090" + path
130         data = None if body is None else json.dumps(body).encode()
131         req = urllib.request.Request(url, data=data, method=method,
headers=headers)
132         with opener.open(req, timeout=15) as resp:
133             raw = resp.read()
134             return json.loads(raw.decode()) if raw else None
135
136     def switch(selector, target):
137         path = "/proxies/" + urllib.parse.quote(selector, safe="")
138         request("PUT", path, {"name": target})
139         print(f"Switched: {selector} -> {target}")
140
141     proxies = request("GET", "/proxies")
142     proxies = proxies.get("proxies", proxies)
143
144     if group not in proxies:
145         raise SystemExit(f"Group not found: {group}")
146
147     if node not in proxies[group].get("all", []):
148         raise SystemExit(f"Node not found in {group}: {node}")
149
150     switch(group, node)
151
152     request("PATCH", "/configs", {"mode": "Global", "ipv6": False})
153     print("Mode switched to Global, IPV6 disabled")
154
155     if "GLOBAL" in proxies:
156         global_all = proxies["GLOBAL"].get("all", [])
157         if node in global_all:
158             switch("GLOBAL", node)
159         elif group in global_all:
160             switch("GLOBAL", group)

```

```

161         else:
162             raise SystemExit("GLOBAL cannot select the target node or group")
163     else:
164         print("GLOBAL selector not found, only mode was switched")
165     PY
166
167     echo "[5/7] Enable shell proxy"
168     source /etc/profile.d/clash.sh
169     proxy_on
170
171     echo "[6/7] Test Docker registry through proxy"
172     HTTP_CODE="$(curl -4 -ss -o /dev/null -w '%{http_code}' \
173         --connect-timeout 20 \
174         -x http://127.0.0.1:7890 \
175         https://registry-1.docker.io/v2/)"
176
177     if [ "$HTTP_CODE" != "401" ]; then
178         echo "Docker registry proxy test failed, HTTP code: $HTTP_CODE"
179         exit 1
180     fi
181
182     echo "Docker registry is reachable through Clash proxy"
183
184     echo "[7/7] Start deployment script in background"
185     cd "$PROJECT_DIR"
186     chmod +x automated-deployment.sh
187
188     nohup bash -lc '
189         set -e
190         source /etc/profile.d/clash.sh
191         proxy_on
192         cd /home/ecs-user/Intelligent_Curtain_Wall
193         ./automated-deployment.sh
194     ' >> "$PROJECT_DIR/deployment.log" 2>&1 &
195
196     echo $! > "$PROJECT_DIR/deployment.pid"
197
198     echo "Deployment script started in background"
199     echo "PID: $(cat "$PROJECT_DIR/deployment.pid")"
200     echo "Log file: $PROJECT_DIR/deployment.log"
201     REMOTE_SCRIPT
202
203     echo "Remote deployment command completed."
204     echo "You can check the deployment logs here:
http://110.42.214.164:9000/deployment-logs"

```


7.3 Secret 配置说明

CI/CD 工作流中不应直接写入密码、密钥、服务器 IP 等敏感信息，应统一配置在 GitHub Secrets 中。

Secret 名称	作用
DOCKER_USERNAME	Docker Hub 用户名
DOCKER_PASSWORD	Docker Hub 密码或 Token
SERVER_HOST	服务器公网 IP
SERVER_SSH_KEY	SSH 私钥
SERVER_PORT	SSH 端口，可选

Table 3

8. 自动化部署脚本

建议把服务器上的部署命令封装成脚本，方便 GitHub Actions 调用。

为减少人工操作，服务器中提供 `deployment.sh` 脚本，用于自动完成镜像拉取、容器更新和无用镜像清理。

8.1 `deployment.sh` 示例

```
1  #!/bin/bash
2  set -e
3  LOG_DIR="/home/ecs-user/Intelligent_Curtain_wall/deployment-logs"
4
5  export TZ="Asia/Shanghai"
6
7  TIMESTAMP=$(date +%Y%m%d-%H:%M)
8  START_TIME=$(date +%Y-%m-%d %H:%M:%S)
9  START_SECONDS=$(date +%s)
10
11
12  mkdir -p "$LOG_DIR"
13
14  {
15      cleanup() {
16          echo "[CLEANUP] Disable proxy and stop clash"
17
18          if [ -f /etc/profile.d/clash.sh ]; then
19              source /etc/profile.d/clash.sh || true
20              proxy_off || true
```

```

21         fi
22
23         sudo pkill -f clash-linux-amd64 || true
24     }
25
26     trap cleanup EXIT
27
28     echo "Execution Start Time: $START_TIME"
29
30     cd /home/ecs-user/Intelligent_Curtain_Wall
31
32     imagesbefore=$(sudo docker images -q --filter "dangling=true")
33     if [ -n "$imagesbefore" ]; then
34         echo "Removing dangling images..."
35         sudo docker rmi $imagesbefore
36     else
37         echo "No dangling images to remove."
38     fi
39
40     sudo docker compose pull
41     sudo docker compose down
42     sudo docker compose up -d
43
44     imagesafter=$(sudo docker images -q --filter "dangling=true")
45     if [ -n "$imagesafter" ]; then
46         echo "Removing dangling images..."
47         sudo docker rmi $imagesafter
48     else
49         echo "No dangling images to remove."
50     fi
51
52     END_TIME=$(date +"%Y-%m-%d %H:%M:%S")
53     END_SECONDS=$(date +%s)
54     ELAPSED_TIME=$((END_SECONDS - START_SECONDS))
55
56     echo "Execution End Time: $END_TIME"
57     echo "Total Execution Time: $ELAPSED_TIME seconds"
58 } &> "$LOG_DIR/$TIMESTAMP.txt"
59

```

Fence 22

赋予执行权限：

```

1 | chmod +x deployment.sh

```

Fence 23

手动执行：

9. Nginx / API Gateway 转发说明

系统使用 Nginx 作为 API Gateway，对外提供统一访问入口。Nginx 根据不同路径将请求转发到对应后端服务，从而避免前端直接访问多个后端端口。

示例配置：

```
1  server {
2      listen 80;
3      server_name _;
4
5      location /api/account/ {
6          proxy_pass http://127.0.0.1:8008;
7          proxy_set_header Host $host;
8          proxy_set_header X-Real-IP $remote_addr;
9      }
10
11     location /api/crack/ {
12         proxy_pass http://127.0.0.1:18001;
13         proxy_set_header Host $host;
14         proxy_set_header X-Real-IP $remote_addr;
15     }
16
17     location /api/stain/ {
18         proxy_pass http://127.0.0.1:18007;
19         proxy_set_header Host $host;
20         proxy_set_header X-Real-IP $remote_addr;
21     }
22
23     location /api/resilience/ {
24         proxy_pass http://127.0.0.1:18005;
25         proxy_set_header Host $host;
26         proxy_set_header X-Real-IP $remote_addr;
27     }
28
29     location /api/vibration/ {
30         proxy_pass http://127.0.0.1:18009;
31         proxy_set_header Host $host;
32         proxy_set_header X-Real-IP $remote_addr;
33     }
34 }
```

修改 Nginx 配置后，需要执行：

```
1 | sudo nginx -t
2 | sudo systemctl reload nginx
```

Fence 26

这里需要特别注意：

如果后端接口本身包含 `/api/account/login`，Nginx 不要随便把 `/api` 前缀删掉，否则可能导致后端路由匹配失败。

10. Docker 调试与故障排查

10.1 查看容器状态

```
1 | docker ps
2 | docker ps -a
3 | docker compose ps
```

Fence 27

10.2 查看容器日志

```
1 | docker logs user-authentication
2 | docker logs -f user-authentication
3 | docker compose logs -f
```

Fence 28

10.3 进入容器内部

```
1 | docker exec -it user-authentication bash
```

Fence 29

如果容器内没有 bash，可以使用：

```
1 | docker exec -it user-authentication sh
```

Fence 30

10.4 测试宿主主机端口

```
1 | curl http://127.0.0.1:8008
2 | curl http://127.0.0.1:18001
```

Fence 31

10.5 测试容器内部服务

```
1 docker exec -it user-authentication sh
2 curl http://127.0.0.1:8000
```

Fence 32

10.6 查看端口占用

```
1 sudo ss -tulpn
2 sudo ss -tulpn | grep 8008
```

Fence 33

10.7 查看容器详细信息

```
1 docker inspect user-authentication
```

Fence 34

10.8 常见问题

问题	可能原因	解决方法
502 Bad Gateway	Nginx 无法连接后端服务	检查容器是否运行、端口是否正确
500 Internal Server Error	后端程序内部异常	查看后端日志
401 Unauthorized	鉴权失败	检查 Token、账号密码、认证逻辑
403 Forbidden	权限不足或接口被拦截	检查 Spring Security / 权限配置
端口无法访问	防火墙或端口未监听	检查 <code>ss -tulpn</code> 和安全组
镜像拉取失败	网络无法访问 Docker Hub	配置镜像源或代理
容器反复重启	程序启动失败或环境变量错误	查看 <code>docker logs</code>

Table 4

11. 回滚方案

当新版本部署后出现严重故障时，可以通过镜像版本回滚恢复服务。

11.1 使用旧版本镜像回滚

修改 `docker-compose.yml` 中的镜像标签，例如：

```
1 image: clearpool/intelligent-curtain-wall:user-authentication-v1.0.0
```

Fence 35

然后执行：

```
1 docker compose pull
2 docker compose up -d
```

Fence 36

11.2 使用 Git 回滚配置文件

```
1 git log --oneline
2 git checkout <commit_id> docker-compose.yml
3 docker compose up -d
```

Fence 37

11.3 回滚注意事项

- 回滚前需要确认数据库结构是否兼容；
- 回滚时应保留当前故障日志；
- 回滚完成后需要重新测试核心接口；
- 不建议直接删除旧镜像，应至少保留最近一个稳定版本。

12. 安全与维护建议

为保证后端服务稳定运行，部署过程中应遵循以下原则：

1. 不在代码仓库中明文存储数据库密码、Token、SSH 私钥等敏感信息；
2. 使用 GitHub Secrets 管理 CI/CD 所需密钥；
3. 数据库端口不直接暴露到公网；
4. 生产环境只开放必要端口；
5. 对 Docker 容器配置 `restart: always`，提高服务可用性；
6. 定期清理无用镜像和日志文件；
7. 对核心服务保留日志，便于故障追踪；
8. 对部署脚本和 Compose 文件进行版本管理；
9. 部署前先在测试环境验证；
10. 更新 Nginx 配置后必须执行 `nginx -t` 检查语法。

13. 附录：常用命令

本节整理后端部署、Docker Compose 管理、容器调试、端口排查和 Nginx 配置检查过程中常用的命令，便于部署和维护人员快速定位问题。

13.1 Docker Compose 常用命令

进入项目部署目录：

```
1 | cd /home/ecs-user/Intelligent_Curtain_wall
```

Fence 38

启动全部服务：

```
1 | docker compose up -d
```

Fence 39

停止全部服务：

```
1 | docker compose down
```

Fence 40

重启全部服务：

```
1 | docker compose restart
```

Fence 41

拉取最新镜像：

```
1 | docker compose pull
```

Fence 42

拉取镜像并重新部署：

```
1 | docker compose pull
2 | docker compose up -d
```

Fence 43

查看 Compose 服务状态：

```
1 | docker compose ps
```

Fence 44

查看全部服务日志：

```
1 | docker compose logs
```

Fence 45

实时查看全部服务日志：

```
1 | docker compose logs -f
```

Fence 46

查看指定服务日志：

```
1 | docker compose logs -f user-authentication
```

Fence 47

重新创建指定服务：

```
1 | docker compose up -d --force-recreate user-authentication
```

Fence 48

停止指定服务：

```
1 | docker compose stop user-authentication
```

Fence 49

启动指定服务：

```
1 | docker compose start user-authentication
```

Fence 50

重启指定服务：

```
1 | docker compose restart user-authentication
```

Fence 51

13.2 Docker 容器常用命令

查看正在运行的容器：

```
1 | docker ps
```

Fence 52

查看所有容器，包括已停止容器：

```
1 | docker ps -a
```

Fence 53

查看容器日志：

```
1 | docker logs user-authentication
```

Fence 54

实时查看容器日志：

```
1 | docker logs -f user-authentication
```

Fence 55

查看最近 100 行日志：

```
1 | docker logs --tail=100 user-authentication
```

Fence 56

进入容器内部：

```
1 | docker exec -it user-authentication bash
```

Fence 57

如果容器内没有 bash，则使用：

```
1 | docker exec -it user-authentication sh
```

Fence 58

查看容器详细信息：

```
1 | docker inspect user-authentication
```

Fence 59

查看容器资源占用：

```
1 | docker stats
```

Fence 60

停止容器：

```
1 | docker stop user-authentication
```

Fence 61

启动容器：

```
1 | docker start user-authentication
```

Fence 62

重启容器：

```
1 | docker restart user-authentication
```

Fence 63

删除已停止容器：

```
1 | docker rm user-authentication
```

Fence 64

13.3 Docker 镜像常用命令

查看本地镜像：

```
1 | docker images
```

Fence 65

拉取指定镜像：

```
1 | docker pull clearpool/intelligent-curtain-wall:user-authentication-new
```

Fence 66

删除指定镜像：

```
1 | docker rmi clearpool/intelligent-curtain-wall:user-authentication-new
```

Fence 67

清理悬空镜像：

```
1 | docker image prune -f
```

Fence 68

清理未使用的镜像、容器和网络：

```
1 | docker system prune -f
```

Fence 69

查看 Docker 磁盘占用：

```
1 | docker system df
```

Fence 70

13.4 端口与网络排查命令

查看所有监听端口：

```
1 | sudo ss -tulpn
```

Fence 71

查看指定端口占用情况：

```
1 | sudo ss -tulpn | grep 8008
```

Fence 72

查看 18001 端口是否监听：

```
1 | sudo ss -tulpn | grep 18001
```

测试本机后端服务是否可访问：

Fence 73

```
1 | curl http://127.0.0.1:8008
```

Fence 74

测试指定接口：

```
1 | curl http://127.0.0.1:8008/api/account/login
```

Fence 75

测试外部访问：

```
1 | curl http://服务器公网IP:8008
```

Fence 76

测试容器内部服务：

```
1 | docker exec -it user-authentication sh
2 | curl http://127.0.0.1:8000
```

Fence 77

查看 Docker 网络：

```
1 | docker network ls
```

Fence 78

查看指定 Docker 网络详情：

```
1 | docker network inspect intelligent_curtain_wall_curtainwall-net
```

Fence 79

13.5 Nginx 常用命令

检查 Nginx 配置语法：

```
1 | sudo nginx -t
```

Fence 80

重新加载 Nginx 配置：

```
1 | sudo systemctl reload nginx
```

Fence 81

重启 Nginx：

```
1 | sudo systemctl restart nginx
```

查看 Nginx 运行状态:

Fence 82

```
1 | sudo systemctl status nginx
```

Fence 83

查看 Nginx 错误日志:

```
1 | sudo tail -f /var/log/nginx/error.log
```

Fence 84

查看 Nginx 访问日志:

```
1 | sudo tail -f /var/log/nginx/access.log
```

Fence 85

查看 Nginx 配置文件:

```
1 | sudo vim /etc/nginx/nginx.conf
```

Fence 86

查看站点配置文件:

```
1 | sudo vim /etc/nginx/sites-available/default
```

Fence 87

13.6 系统服务与防火墙命令

查看 Docker 服务状态:

```
1 | sudo systemctl status docker
```

Fence 88

重启 Docker 服务:

```
1 | sudo systemctl restart docker
```

Fence 89

设置 Docker 开机自启:

```
1 | sudo systemctl enable docker
```

Fence 90

查看防火墙状态:

```
1 | sudo ufw status
```

Fence 91

开放指定端口：

```
1 | sudo ufw allow 8008/tcp
```

Fence 92

开放多个后端端口示例：

```
1 | sudo ufw allow 18001/tcp
2 | sudo ufw allow 18005/tcp
3 | sudo ufw allow 18007/tcp
4 | sudo ufw allow 18009/tcp
```

Fence 93

重新加载防火墙：

```
1 | sudo ufw reload
```

Fence 94

13.7 常用健康检查命令

检查用户认证服务：

```
1 | curl http://127.0.0.1:8008
```

Fence 95

检查裂缝检测服务：

```
1 | curl http://127.0.0.1:18001
```

Fence 96

检查韧性评估服务：

```
1 | curl http://127.0.0.1:18005
```

Fence 97

检查污渍检测服务：

```
1 | curl http://127.0.0.1:18007
```

Fence 98

检查振动检测服务：

```
1 | curl http://127.0.0.1:18009
```

Fence 99

13.8 部署脚本执行命令

进入脚本目录：

```
1 | cd /home/ecs-user/Intelligent_Curtain_wall/scripts
```

Fence 100

赋予脚本执行权限：

```
1 | chmod +x deploy.sh
```

Fence 101

执行部署脚本：

```
1 | ./deploy.sh
```

Fence 102

后台执行部署脚本并保存日志：

```
1 | nohup ./deploy.sh > deploy.log 2>&1 &
```

Fence 103

查看部署日志：

```
1 | tail -f deploy.log
```

Fence 104

13.9 常见故障快速定位命令

容器是否运行：

```
1 | docker ps -a
```

Fence 105

服务是否监听端口：

```
1 | sudo ss -tulpn | grep 端口号
```

Fence 106

容器日志是否报错：

```
1 | docker logs --tail=100 容器名称
```

Fence 107

Nginx 配置是否正确：

```
1 | sudo nginx -t
```

Nginx 是否正常运行:

Fence 108

```
1 | sudo systemctl status nginx
```

Fence 109

服务器磁盘是否占满:

```
1 | df -h
```

Fence 110

查看内存占用:

```
1 | free -h
```

Fence 111

查看 CPU 和进程情况:

```
1 | top
```

Fence 112

查看系统日志:

```
1 | journalctl -xe
```

Fence 113

查看 Docker 服务日志:

```
1 | journalctl -u docker -f
```

Fence 114